

Three use cases we need asynchronous programming

1. I/O-bound operations

- When your program needs to read or write data to external resources like:
 - Databases
 - File systems
 - Network requests (APIs, HTTP calls)
- Example: Fetching data from a web API without blocking the main thread.
- **Benefit:** The program can continue executing other tasks while waiting for the I/O operation to complete

2. User interface responsiveness

- In GUI applications (desktop, mobile, or web), long-running operations like loading files or downloading data can freeze the UI if done synchronously.
- Example: In a WPF or WinForms app, loading a large image asynchronously keeps the UI responsive.
- **Benefit:** Users can interact with the application while background tasks are running.

3. Parallel processing of multiple tasks

- When you have multiple independent tasks that can run at the same time.
- Example: Sending multiple HTTP requests to different servers simultaneously or processing large datasets in chunks asynchronously.
- **Benefit:** Reduces total execution time and improves resource utilization.

What is the difference between thread and task?

Thread

- **Definition:** A thread is a low-level unit of execution in a process.
- **Managed by:** The operating system.
- **Creation:** You explicitly create and start threads using **Thread** class.

Characteristics:

- **Manual management:** you start, stop, and join threads yourself.
- **Threads are heavyweight,** meaning they consume more system resources.
- **Good for long-running, independent background operations.**
- **Harder to handle errors and cancellations.**

Task

- **Definition:** A task is a higher-level abstraction over threads that represents an asynchronous operation.
- **Managed by:** The .NET Task Parallel Library (TPL) and thread pool.
- **Creation:** Usually with `Task.Run` or `async/await`.

Characteristics:

- **Easier to work with:** supports `async/await`, cancellation tokens, and continuations.
- Tasks are lightweight and can reuse threads from the thread pool.
- **Good for I/O-bound or parallelizable operations.**
- Automatic error handling and continuation chaining.

What are delegate built_in types in detail?

What is a delegate?

- **Definition:** A delegate is a type that represents references to methods.
- Think of it as a type-safe function pointer in C#.
- You can store methods in a delegate and call them later.

Built-in delegate types in C#

C# has three main built-in generic delegates, so you don't need to define custom delegates for common patterns:

a) `Func<>`

- Represents a method that returns a value.
- The last type parameter is the return type, others are input parameters.

b) `Action<>`

- Represents a method that does not return a value (`void` methods).
- Can take 0 or more input parameters.

c) `Predicate<T>`

- Represents a method that takes one input and returns a boolean.
- Often used in searching, filtering, and conditions.